

Physarum-Inspired Branch-Flow Search

The machine-checked core of a self-organizing game-tree search

Abstract

Branch-flow search assigns computational traffic to complete root-to-frontier branches of a game tree, and lets useful traffic reinforce every edge on the branch it travelled. The substrate is a conductivity network borrowed from the slime mold *Physarum polycephalum*. A move policy supplies only the initial conductivities, and no learned action decides where the next unit of computation goes.

This document states what has been proved about that algorithm and nothing else. The Lean 4 development contains forty-two theorems. Thirty-eight are checked by the Lean kernel against its three standard axioms. Four are exact-rational computations discharged by `native_decide`, which also trusts the Lean compiler, and they are marked wherever they appear. Section 10 lists what the formalization does not answer, including every question about playing strength.

1 Scope

The repository holds one search algorithm. A separate line of work on learned computation-budget allocation is not included here.

The theorems cover the algebra of the search dynamics: how conductivities are initialized from a policy, how branch traffic is credited, how unused routes decay, and how cost accumulates. They hold for any flow assignment and any usefulness signal meeting the stated sign conditions. That generality is what makes them cheap to state, and it is also what limits them.

Section 7 removes part of that generality by constructing the map $q = F(D)$ from conductivities to branch flows on a tree, and by proving conservation and coupling for it. The two halves are not yet joined: the credit theorems still quantify over an arbitrary flow rather than over the flows F produces. No theorem here says the search plays well.

2 Where the idea comes from

Physarum polycephalum is an acellular slime mold. It feeds as one multinucleate cell spread into a network of protoplasmic tubes, and it has no brain and no central controller. It still solves shortest-path problems, by a local rule: tubes carrying a large flux of protoplasm thicken, and tubes carrying little flux thin out and disappear.

Nakagaki, Yamada and Tóth showed the organism connecting two food sources through the shortest route in a maze [1]. Tero, Kobayashi and Nakagaki then modelled the mechanism as a conductivity network, where each tube has a conductivity D_e obeying

$$\frac{dD_e}{dt} = f(|Q_e|) - D_e, \tag{1}$$

with Q_e the flux through the tube under a Kirchhoff flow and f increasing [2]. Later work reproduced the Tokyo rail network from the same rule [3].

What is borrowed here is the feedback loop,

$$\text{conductivity} \rightarrow \text{flow} \rightarrow \text{reinforcement} \rightarrow \text{conductivity}. \tag{2}$$

The network is never designed. It is what the traffic left behind.

3 What changes when the substrate is a game tree

A slime mold reinforces a tube in proportion to the flux through it. Flux alone is the wrong signal for game-tree search, because a search route can carry heavy traffic and learn nothing. So the reinforcement is gated by an observed usefulness U_b , measured after the branch has been searched rather than predicted before it.

Two further commitments separate the model from ordinary tree search.

The unit of traffic is a complete branch. A branch b is a visited path from the root to a frontier or terminal node, with edge set P_b . A deep observation is evidence about every prefix decision that made the observation reachable, so one observation credits every edge on its route in a single event.

The policy head sets initial conditions only. An ordinary move policy $\pi_\theta(a \mid s)$ supplies a branch mass

$$m_\theta(b) = \prod_{k=1}^d \pi_\theta(a_k \mid s_k), \quad (3)$$

which initializes conductivity on newly represented edges, above a positive exploration floor ε . The policy does not select which computation happens next, and it does not fix the final topology.

The discrete update is

$$D_e^{t+1} = \rho D_e^t + \alpha \sum_{b: e \in P_b} |q_b^t| U_b^t, \quad 0 < \rho < 1. \quad (4)$$

Sections 5 through 9 state what is proved about this update.

4 The formal model

The development works over arbitrary finite types **Edge** and **Branch** with rational arithmetic. Exact rationals are used throughout, so the executable diagnostic in Section 9 has no floating-point slack.

A branch is represented by its edge set, **path** : **Branch** → **Finset Edge**. The model fixes the bookkeeping of flow, credit and decay without committing to how q_b and U_b are computed. The four definitions that carry the model are these.

```
def branchPrior (localProbabilities : List ℚ) : ℚ :=
  localProbabilities.prod

noncomputable def policyConductivity
  (path : Branch → Finset Edge) (branchMass : Branch → ℚ)
  (explorationFloor : ℚ) (edge : Edge) : ℚ :=
  explorationFloor +
    ∑ branch, if edge ∈ path branch then branchMass branch else 0

noncomputable def depositedTraffic
  (path : Branch → Finset Edge) (flow usefulness : Branch → ℚ)
  (edge : Edge) : ℚ :=
  ∑ branch, if edge ∈ path branch then flow branch * usefulness branch else 0

noncomputable def evolve
  (path : Branch → Finset Edge) (flow usefulness : Branch → ℚ)
  (retention learningRate : ℚ) (conductivity : Edge → ℚ)
  (edge : Edge) : ℚ :=
  retention * conductivity edge +
    learningRate * depositedTraffic path flow usefulness edge
```

`evolve` is equation (4), with `retention` for ρ and `learningRate` for α .

5 Policy initialization

`branchPrior_nil` gives the empty branch prior 1.

`branchPrior_append` gives $m(u \mathbin{++} v) = m(u)m(v)$. Prefix masses compose, so an edge created during search can be initialized from its prefix without enumerating the tree.

`branchPrior_extension_le`: if $m(u) \geq 0$ and the appended probability lies in $[0, 1]$, then $m(u \mathbin{++} [p]) \leq m(u)$. Branch priors are non-increasing in depth. This is the reason a product prior on its own cannot carry a deep line, it decays geometrically, and the exploration floor rather than the prior is what keeps deep edges alive.

`policyConductivity_pos`: for a strictly positive floor $\varepsilon > 0$ and nonnegative branch masses, every initialized edge has $D_e^0 > 0$.

That positivity result does real work. If flow is proportional to conductivity, an edge initialized at exactly zero carries no flow, so it receives no deposit, so it stays at zero. A policy assigning probability zero to a move would then delete that move from the search permanently. The floor is what keeps the policy a prior instead of a constraint.

6 Flow bookkeeping

`branchFlow_conservation_at_split` takes `active : Finset Branch`, the branches through an internal node, a function `nextChild : Branch → Child` giving the child each one continues through, and `flow : Branch → ℚ`. It states

$$\sum_c \left(\sum_{b \in \text{active}} [\text{nextChild}(b) = c] q_b \right) = \sum_{b \in \text{active}} q_b. \quad (5)$$

The hypothesis is worth stating plainly: each active branch continues through exactly one child, which is what `nextChild` being a function encodes. The theorem is then the re-indexing identity that outgoing child traffic sums to incoming traffic. It is conservation in the bookkeeping sense. It is not a derivation of Kirchhoff's law from a graph Laplacian, and it does not show that any particular flow solver produces a conserved field.

7 The flow map

Everything above takes `flow` as a free function. Nothing ties it to the conductivities, so those theorems hold for flow assignments that no conductivity field could produce. `TrickyProof/FlowSolver.lean` supplies the map $q = F(D)$.

On a tree the electrical problem is series-parallel, so F has a closed form and needs no matrix inversion. Write $R(v)$ for the resistance from v to ground and s_e for the conductance of edge e in series with the subtree under it. Then

$$s_e = \frac{1}{1/D_e + R(\text{head } e)}, \quad R(v) = \frac{1}{\sum_{e \in \text{out}(v)} s_e}, \quad R(\text{sink}) = 0, \quad (6)$$

and current entering a node splits between its children in proportion to s_e . Composing the divider fractions down a branch gives

$$q_b = I_0 \prod_{k=1}^d \sigma_{e_k}, \quad \sigma_e = \frac{s_e}{\sum_{e' \in \text{out}(\text{tail } e)} s_{e'}}. \quad (7)$$

This is the structural analogue of the policy product $m_\theta(b) = \prod_k \pi_\theta(a_k \mid s_k)$, with one difference that matters. Each σ_e depends on $R(\text{head } e)$, so it depends on the whole subtree below the edge. The divider fractions are not local, and that non-locality is the coupling the design claims over per-edge counters.

`sinkFlows_sum` states that all injected current reaches the sinks. It is the conservation law that `branchFlow_conservation_at_split` assumes, derived here from the definition of the solver, so it constrains F rather than describing an arbitrary flow.

`series_strictAnti_resistance` and `sibling_starvation` give the coupling, and `subtree_reinforcement_starves_sibling` combines them: lowering the resistance inside one subtree, without touching that subtree’s root edge, strictly reduces a sibling’s flow.

`reinforcing_A_starves_B_and_C` is an exact instance. The tree is

```

root --1--> v --dA--> sink      (branch A)
              v --dB--> sink      (branch B)
root --1--> sink                (branch C)

```

At $d_A = d_B = 1$ the sink flows are $[1/5, 1/5, 3/5]$. Reinforcing A to $d_A = 9$ gives $[3/7, 1/21, 11/21]$. B falls, which is expected because it shares the prefix. C falls too, from $3/5$ to $11/21$, and C shares no edge with A. Incrementing independent per-edge counters cannot move C.

The scope is trees. On a transposition DAG the Laplacian still determines the edge flow Q_e , but $Q_e = \sum_{b \ni e} q_b$ has many solutions, so the branch flows need an extra convention and the deposit rule inherits it.

8 Whole-branch credit, decay, and amplification

The single-event form isolates one transported branch.

```

def reinforceBranch (path : Finset Edge)
  (retention learningRate flow usefulness : ℚ)
  (conductivity : Edge → ℚ) (edge : Edge) : ℚ :=
  retention * conductivity edge +
  if edge ∈ path then learningRate * flow * usefulness else 0

```

`reinforceBranch_on_path`: every edge on the branch gets the same direct credit, whatever its depth,

$$D'_e - \rho D_e = \alpha q u \quad \text{for all } e \in P_b. \quad (8)$$

`reinforceBranch_off_path`: every edge off the branch gets decay and nothing else, $D'_e = \rho D_e$.

`whole_branch_credit`, summing over the branch,

$$\sum_{e \in P_b} (D'_e - \rho D_e) = |P_b| \cdot \alpha q u. \quad (9)$$

`total_depositedTraffic` is the many-branch version, by finite double counting,

$$\sum_e H_e = \sum_b |P_b| q_b u_b, \quad H_e = \sum_{b: e \in P_b} q_b u_b. \quad (10)$$

The factor $|P_b|$ in (9) is easy to misread. A longer branch absorbs proportionally more total deposit, and that is an accounting consequence of crediting every edge equally, not evidence that long branches carry more information. If the length preference is unwanted, replace U_b by $U_b/|P_b|$, or divide by measured branch cost. What the theorem is good for is forcing that choice into the open instead of leaving it inside a heuristic.

depositedTraffic_nonnegative: nonnegative flow and nonnegative usefulness give $H_e \geq 0$.

evolve_positive: with $\rho > 0$, $\alpha \geq 0$, all conductivities positive, and nonnegative flow and usefulness, **evolve** keeps every conductivity strictly positive. The update never drives an edge to zero, so decay suppresses a route instead of deleting it, and a faded route stays revivable.

unused_edge_decays: for $D_e > 0$ and $\rho < 1$, an edge receiving no deposit satisfies $\rho D_e < D_e$ strictly.

larger_gain_amplifies_relative_conductivity: writing one step in multiplicative form $D'_e = (\rho + g_e)D_e$, if $D_A, D_B > 0$ and $g_B < g_A$ then

$$D'_B D_A < D'_A D_B. \quad (11)$$

This is the positive-feedback law. It is stated by cross-multiplication, so no division and no nonzero-denominator side condition appears, and the common retention term drops out. The edge with the larger useful-flow gain gains relative conductivity. It is ordered-field algebra and assumes nothing about where g_A and g_B came from, so it does not say a larger gain corresponds to a better move.

9 Budget as integrated traffic, and the exact diagnostic

There is no learned action that spends a unit of budget. Computation is consumed by transport and evaluation, so the budget is an integral,

$$B_n = \sum_{t < n} \sum_b c_b^t |q_b^t|. \quad (12)$$

usedBudget_append_round: appending a round adds its cost.

usedBudget_monotone: with nonnegative round costs, accumulated budget is non-decreasing. Stopping at a measured cost is therefore well defined, and the procedure is anytime.

The two-branch diagnostic

BranchFlowMain.lean runs the dynamics on the smallest configuration that can show the intended behaviour. Branch A is two unit-length edges deep, branch B is one edge, and the policy prefers B. Retention is $\rho = 3/4$, usefulness is $U_A = 4$ against $U_B = 1/2$, and all arithmetic is over $\mathbb{Q}$.

```
round=0, D={ aRoot := 2/5,   aDeep := 2/5,   bRoot := 3/5   }, flow(A)=1/5,   flow(B)=3/5,   cost=1
round=1, D={ aRoot := 11/10, aDeep := 11/10, bRoot := 3/4   }, flow(A)=11/20, flow(B)=3/4,   cost=37/20
round=2, D={ aRoot := 121/40, aDeep := 121/40, bRoot := 15/16 }, flow(A)=121/80, flow(B)=15/16, cost=317/80
```

At the start $1/5 < 3/5$, so the policy's preferred shallow branch carries three times the traffic. The first useful observation on A credits both of its edges equally and raises each to $11/10$. By round two the ordering has reversed,

$$\frac{121}{80} > \frac{15}{16}, \quad (13)$$

and the deeper branch that the policy disfavoured carries more flow. Four theorems pin these exact rationals:

```
initial_policy_prefers_b
first_useful_observation_credits_both_a_edges
useful_deep_branch_eventually_carries_more_flow
initial_round_cost
```

This shows that whole-branch reinforcement can overturn a policy initialization, on one configuration, with usefulness values chosen by hand rather than measured from a game tree. It says the mechanism does what it is described as doing. It says nothing about search quality.

These four theorems use **native_decide**. See Section 11.

10 What is not proved

The gap between the theorems above and a working engine is the whole engineering problem, so it is listed directly.

- Nothing about playing strength. No theorem compares branch-flow search with alpha-beta, PUCT MCTS, best-first policy search, or random frontier expansion. Whether the coupled flow dynamics beat any of them is empirical and is not settled here.
- Nothing about the usefulness function. U_b is an arbitrary nonnegative rational in every theorem. That a particular U correlates with playing strength, or resists manipulation, is unproved, and it is the main place where the design can fail.
- Nothing about convergence. There is no theorem that the conductivity field converges, that it concentrates on principal variations, or that the induced root decision approaches the minimax decision.
- The flow solver covers trees only. Section 7 formalizes F and derives conservation and coupling, but on a transposition DAG the branch decomposition is not unique, and no result bounds the cost of the solve, which the loop pays every round out of the same budget.
- The two halves are not yet joined. **BranchFlow** quantifies over an arbitrary **flow** and **FlowSolver** constructs a particular one, and nothing instantiates the first with the second. The credit theorems have not been specialized to flows that F can produce.
- No adversarial backup. Minimax, alpha-beta bounds, and the separation between the flow potential p_v and the game value $V(v)$ are part of the model but are not formalized here.
- Nothing about the length preference. The $|P_b|$ factor in (9) is proved to exist. Whether it helps or hurts is not addressed.
- No claim of novelty over ant-colony or MCTS variants. If the coupled flow solve is approximated by sampling one path per round and incrementing edge counts, the algorithm degenerates toward existing methods. The theorems here hold in both cases, so they do not distinguish them.

11 Trusted computing base

The thirty-eight library theorems, in **BranchFlow.lean** and **FlowSolver.lean**, depend on Lean’s three standard axioms and nothing else:

```
propext, Classical.choice, Quot.sound
```

These are the axioms of classical higher-order logic as used throughout Mathlib. Trusting these results means trusting the Lean kernel, those axioms, and the statements being the intended ones.

The four theorems in **BranchFlowMain.lean** each carry one extra axiom, of the form

```
BranchFlowDemo.<name>._native.native_decide.ax_1_1
```

native_decide evaluates a decidable proposition with compiled code and asserts the result, so it trusts the Lean compiler and runtime on top of the kernel. That is a strictly larger trusted base. The claims affected are only the exact rationals in the Section 9 table. Replacing **native_decide** with kernel-level **decide** or **norm_num** would remove the dependency, and is worth doing.

The audit reproduces in full:

```
lake build
lake env lean AxiomAudit.lean
```

There are no occurrences of `sorry` or `admit` in the repository.

12 Reproducing

```
lake exe cache get      # optional: prebuilt Mathlib oleans
lake build              # library and diagnostic
lake exe branch_flow_demo
lake env lean AxiomAudit.lean
```

Toolchain is `leanprover/lean4:v4.33.1` with Mathlib `v4.33.1`.

13 Summary

The claim supported here is narrow and should be read narrowly. Given any nonnegative flow assignment and any nonnegative usefulness signal, whole-branch reinforcement with common decay is coherent: initialization stays positive above a floor, credit reaches every depth of a transported branch in one event, unused routes strictly decay without being deleted, larger useful gains strictly amplify relative conductivity, accumulated cost is monotone, and on one exact two-branch configuration the dynamics reverse a policy initialization in two rounds.

Coherence is a precondition. It is not a result about search quality, and nothing above should be read as one. Whether the loop

$$\text{policy prior} \rightarrow \text{branch flow} \rightarrow \text{information} \rightarrow \text{conductivity} \rightarrow \text{new branch flow} \quad (14)$$

searches a game tree well is still open. The decisive test is likely the ablation that removes branch-wide credit in favour of endpoint-only credit. If crediting the whole transported branch does not improve regret per unit of cost, the main motivation for the design is absent.

References

- [1] T. Nakagaki, H. Yamada, Á. Tóth. *Maze-solving by an amoeboid organism*. Nature **407**, 470 (2000).
- [2] A. Tero, R. Kobayashi, T. Nakagaki. *A mathematical model for adaptive transport network in path finding by true slime mold*. Journal of Theoretical Biology **244**(4), 553–564 (2007).
- [3] A. Tero, S. Takagi, T. Saigusa, K. Ito, D. P. Bebber, M. D. Fricker, K. Yumiki, R. Kobayashi, T. Nakagaki. *Rules for biologically inspired adaptive network design*. Science **327**(5964), 439–442 (2010).